

Mental Health at Work: Associations Between Workplace Policy, Culture, and Disclosure in Tech

Maria A. Suarez



I. Introduction & Business Understanding

Problem (why this matters)

Many tech workers need mental-health support, but **they don't feel safe telling work**—especially when **leave is hard** and **anonymous help isn't clear**, and **managers feel less safe than coworkers**.



Question?

Which workplace factors (e.g., company size, available benefits, gender, remote work status) are associated with whether an employee seeks treatment for a mental health condition?

Goal

Goal: Prioritize supportive outreach using OSMI survey signals

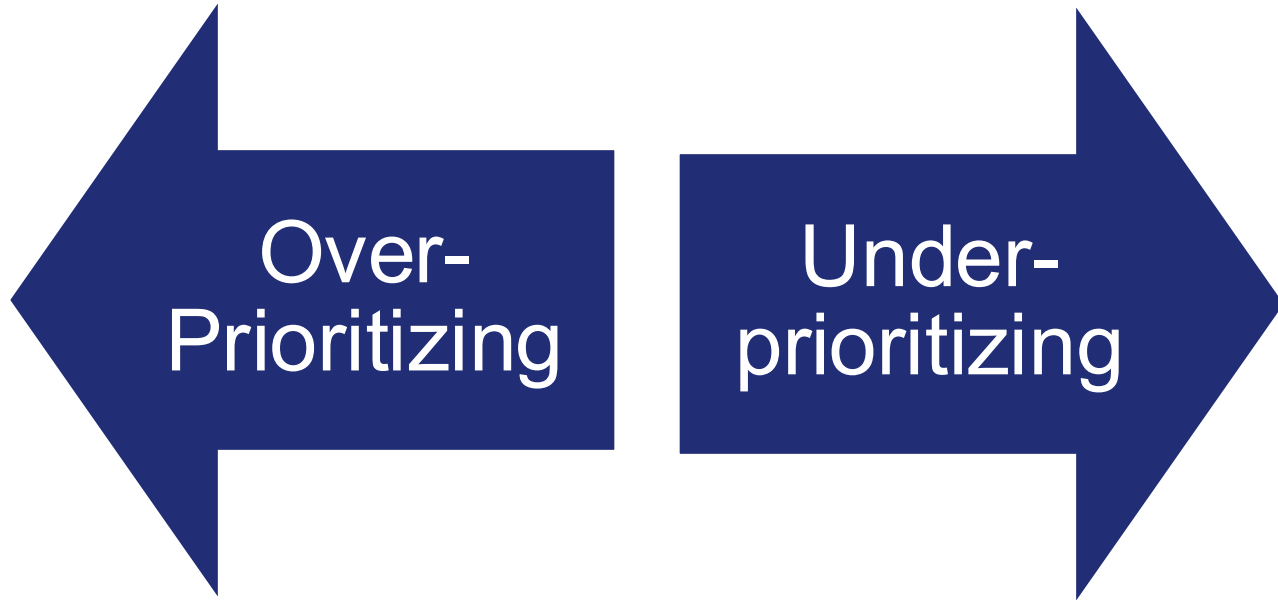
STIGMA
SCORES

LEAVE POLICY
CLARITY

ANONYMITY

BENEFITS

Decision: Who should HR proactively check in with first?



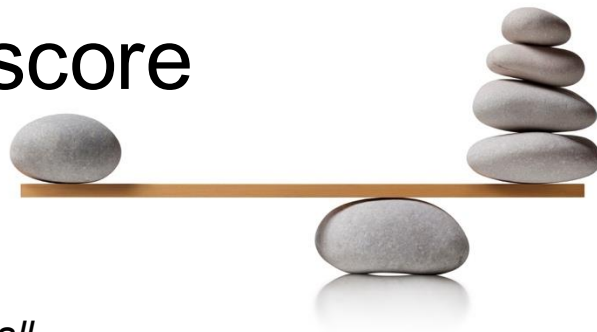
Metrics

- F1 Score: “Fairness” score

- Recall: Are we catching people who really might need help?
- Precision: Of the people we flag, how many truly at-risk (not false alarms)?

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

- PR-AUC (Precision-Recall AUC)



Imbalance & Why F1/PR-AUC (says it early)

Target is imbalanced, so accuracy can be misleading.

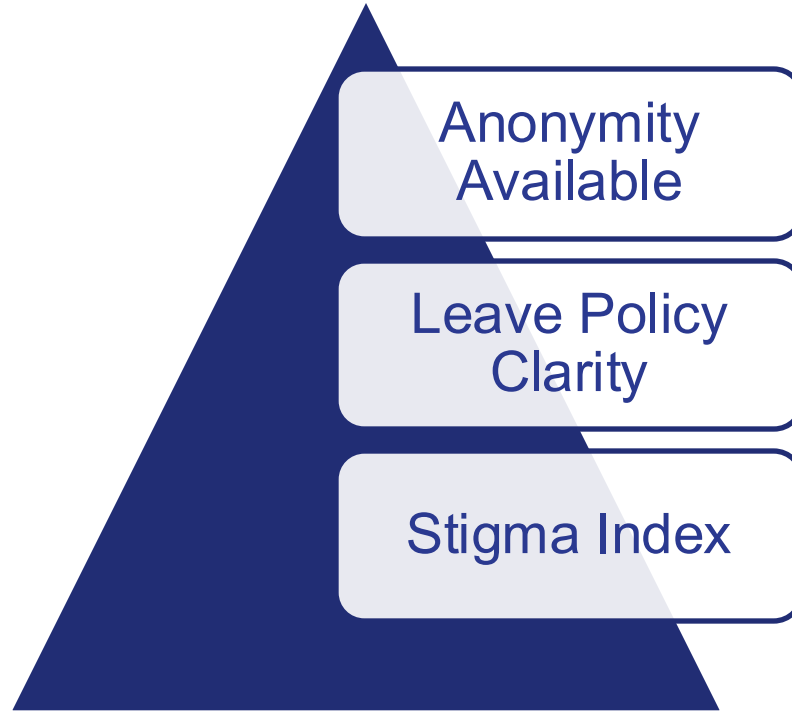


We optimize F1 (balance of precision/recall) and track PR-AUC for rare positives.



Result: LogReg beat RF on F1/recall; ROC-AUC strong too (≈ 0.81 vs 0.71)

Data that drives the score



Hypotheses

- Higher stigma relates to lower disclosure / higher risk
- Clear leave policies and anonymity correlate with safer outcomes
- Recent related responses are predictive of disclosure decisions

Variables & Targets

Variable	IV/DV	Type	Why it's included / What it means
Disclosure / Risk Label	DV	Binary (0/1)	: engineered from survey responses; 1 = at-risk / ITargetikely not to disclose
stigma_index	IV	Numeric (composite)	Higher stigma → higher risk
leave_policy_clarity	IV	Ordinal (low→high)	Clearer leave policy → lower risk
anonymity_available	IV	Binary (yes/no)	Anonymous channel available → lower risk
benefits	IV	Binary/Categorical	Mental-health benefits offered
care_options_known	IV	Binary	Knows how to access care → lower risk
seek_help_comfort	IV	Ordinal (low→high)	Comfort seeking help → lower risk
company_size	IV	Categorical (one-hot)	Organizational context / resources
age_bucket	IV	Categorical (one-hot)	Demographic segment (e.g., 18–24, 25–34, ...)
gender	IV	Categorical (one-hot)	Demographic segment
— Engineered: ordinal encodings	IV	Ordinal numeric	Preserve order for policy/comfort scales
— Engineered: one-hot encodings	IV	One-hot dummies	Model-ready encoding for categories

II. Data Acquisition

Dataset-Mental Health in Tech Survey

Source: *Open Sourcing Mental Illness (OSMI)* – an annual survey of tech workers' attitudes and experiences with mental health in the workplace (2014-2016).

Electronic, anonymous/consented responses; workplace policy & attitude coverage

Sample size after cleaning: ~1,259 respondents.

Format: CSV survey file.

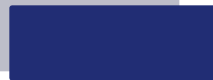
https://www.kaggle.com/datasets/osmi/mental-health-in-tech-survey?utm_source=chatgpt.com

Dataset-Mental Health in Tech Survey



Rich context;
widely used;
accessible

Self-report
bias; non-
probability
sample;
varying yearly
design



Dataset-Key Features

Demographics: age, gender, company size.

Workplace culture: anonymity, leave policy, mental health benefits, care options.

Personal factors: willingness to seek help, perceptions of stigma (stigma index).

Target variable: *Disclosure* → “Would you disclose a mental health condition at work?” (Yes/No).

III. Data Management

- survey.csv

Data was
already in CSV
format.

Load the raw CSV

```
In [6]: df = pd.read_csv("survey.csv")
print("Shape (rows,cols):", df.shape)
print("\nColumn names (first 20):\n", list(df.columns[:20]))
df.head(10)
```

Shape (rows,cols): (1259, 27)

Column names (first 20):

['Timestamp', 'Age', 'Gender', 'Country', 'state', 'self_employed', 'family_history', 'treatment', 'work_interfere', 'no_employees', 'remote_work', 'tech_company', 'benefits', 'care_options', 'wellness_program', 'seek_help', 'anonymity', 'leave', 'mental_health_consequence', 'phys_health_consequence']

```
Out[6]:
```

	Timestamp	Age	Gender	Country	state	self_employed	family_history	treatment	work_interfere	no_employees	...	le
0	2014-08-27 11:29:31	37	Female	United States	IL	NaN	No	Yes	Often	6-25	...	Somev
1	2014-08-27 11:29:37	44	M	United States	IN	NaN	No	No	Rarely	More than 1000	...	Ck
2	2014-08-27 11:29:44	32	Male	Canada	NaN	NaN	No	No	Rarely	6-25	...	Somev diff
3	2014-08-27 11:29:46	31	Male	United Kingdom	NaN	NaN	Yes	Yes	Often	26-100	...	Somev diff
4	2014-08-27 11:30:22	31	Male	United States	TX	NaN	No	No	Never	100-500	...	Ck
5	2014-08-27 11:31:22	33	Male	United States	TN	NaN	Yes	No	Sometimes	6-25	...	Ck
6	2014-08-27 11:31:50	35	Female	United States	MI	NaN	Yes	Yes	Sometimes	1-5	...	Somev diff
7	2014-08-27 11:32:05	39	M	Canada	NaN	NaN	No	No	Never	1-5	...	Ck
8	2014-08-27 11:32:39	42	Female	United States	IL	NaN	Yes	Yes	Sometimes	100-500	...	' diff
9	2014-08-27 11:32:43	23	Male	Canada	NaN	NaN	No	No	Never	26-100	...	Ck

IV. Data Cleaning

Data Wrangling-Preview Clean Data Set

```
print("Preview of cleaned dataset:")
display(df.head(10))

# Show shape (rows, columns)
print("\nDataset shape:", df.shape)

# Check if any missing values remain
print("\nRemaining missing values by column:")
print(df.isna().sum()[df.isna().sum() > 0])
```

Preview of cleaned dataset:

	timestamp	age	gender	country	state	self_employed	family_history	treatment	work_interfere	no_employees	...	Sort
0	2014-08-27 11:29:31+00:00	37.0	Female	United States	IL	NaN	No	Yes	Often	6-25	...	Sort
1	2014-08-27 11:29:37+00:00	44.0	Male	United States	IN	NaN	No	No	Rarely	More than 1000	...	
2	2014-08-27 11:29:44+00:00	32.0	Male	Canada	NaN	NaN	No	No	Rarely	6-25	...	Sort
3	2014-08-27 11:29:46+00:00	31.0	Male	United Kingdom	NaN	NaN	Yes	Yes	Often	26-100	...	Sort
4	2014-08-27 11:30:22+00:00	31.0	Male	United States	TX	NaN	No	No	Never	100-500	...	
5	2014-08-27 11:31:22+00:00	33.0	Male	United States	TN	NaN	Yes	No	Sometimes	6-25	...	
6	2014-08-27 11:31:50+00:00	35.0	Female	United States	MI	NaN	Yes	Yes	Sometimes	1-5	...	Sort
7	2014-08-27 11:32:05+00:00	39.0	Male	Canada	NaN	NaN	No	No	Never	1-5	...	
8	2014-08-27 11:32:39+00:00	42.0	Female	United States	IL	NaN	Yes	Yes	Sometimes	100-500	...	
9	2014-08-27 11:32:43+00:00	23.0	Male	Canada	NaN	NaN	No	No	Never	26-100	...	

10 rows x 27 columns

Missing Values Treatment

```
# 1. Drop columns with too many missing values
# Let's say > 40% missing is too much for this project
threshold = 0.4
cols_to_drop = [c for c in df.columns if df[c].isna().mean() > threshold]

print("Dropping columns due to high missingness:", cols_to_drop)
df = df.drop(columns=cols_to_drop)

# 2. Impute numeric columns (median)
num_cols = df.select_dtypes(include=["float64", "int64"]).columns
for c in num_cols:
    median_val = df[c].median()
    df[c] = df[c].fillna(median_val)
    print(f"Filled NaN in numeric column '{c}' with median={median_val}")

# 3. Impute categorical columns (mode or 'Unknown')
cat_cols = df.select_dtypes(include="object").columns
for c in cat_cols:
    if df[c].isna().sum() > 0:
        mode_val = df[c].mode(dropna=True)
        if not mode_val.empty:
            fill_val = mode_val[0] # most frequent value
        else:
            fill_val = "Unknown"
        df[c] = df[c].fillna(fill_val)
        print(f"Filled NaN in categorical column '{c}' with '{fill_val}'")

# Quick check - confirm no missing left
print("\nRemaining missing values:", df.isna().sum().sum())
```

```
Dropping columns due to high missingness: ['state', 'comments']
Filled NaN in numeric column 'age' with median=31.0
Filled NaN in categorical column 'self_employed' with 'No'
Filled NaN in categorical column 'work_interfere' with 'Sometimes'
```

```
Remaining missing values: 0
```

- If a column has **>40% missing**, drop it (cols_to_drop).

- **Numeric Impute (per column):** Replace NaNs with that column's **median** (robust to outliers).

- **Categorical Impute (per column):** Replace NaNs with the **mode** (most frequent value). If no mode exists, use "Unknown".

Missing Values

```
In [12]: na_counts=df.isna().sum().sort_values(ascending=False)
na_pct = (na_counts / len(df) * 100).round(2)
missing_table = pd.DataFrame({"missing_count": na_counts, "missing_pct": na_pct})
missing_table.head(20)
```

```
Out[12]:
```

	missing_count	missing_pct
comments	1095	86.97
state	515	40.91
work_interfere	264	20.97
self_employed	18	1.43
seek_help	0	0.00
obs_consequence	0	0.00
mental_vs_physical	0	0.00
phys_health_interview	0	0.00
mental_health_interview	0	0.00
supervisor	0	0.00
coworkers	0	0.00
phys_health_consequence	0	0.00
mental_health_consequence	0	0.00
leave	0	0.00
anonymity	0	0.00
timestamp	0	0.00
wellness_program	0	0.00
age	0	0.00
benefits	0	0.00
tech_company	0	0.00

I computed how many missing values each column has and what percentage of the dataset that represents. This guides which fields need cleaning or can be safely dropped later.

Standardize column names

```
8]: original_columns=df.columns.tolist()
def clean_colname(c):
    c=str(c).strip().lower()
    c=c.replace("/", " ").replace("\\n", " ").replace("-", " ")
    c=" ".join(c.split())
    c=c.replace("%", "pct")
    c = "".join(ch if ch.isalnum() or ch==" " else "_" for ch in c)
    c = "_".join(c.split())
    while "_" in c:
        c = c.replace("__", "_")
    return c
df.columns = [clean_colname(c) for c in df.columns]

comparison = pd.DataFrame({
    "original": original_columns[:25],
    "cleaned": df.columns[:25]
})
comparison
```

```
8]:
```

	original	cleaned
0	Timestamp	timestamp
1	Age	age
2	Gender	gender
3	Country	country
4	state	state
5	self_employed	self_employed
6	family_history	family_history
7	treatment	treatment
8	work_interfere	work_interfere
9	no_employees	no_employees

I made column names consistent (lowercase, underscores, no special characters). This makes later code cleaner and less error-prone.

Normalize common Yes/No fields

```
: yn_map = {
    "yes": "Yes", "y": "Yes", "true": "Yes", "1": "Yes",
    "no": "No", "n": "No", "false": "No", "0": "No"
}

def normalize_yes_no(val):
    if pd.isna(val):
        return np.nan
    s = str(val).strip().lower()
    return yn_map.get(s, val)

candidate_cols = []
for c in df.columns:
    if df[c].dtype == "O":
        vals = df[c].dropna().astype(str).str.lower().unique()
        if len(vals) <= 6 and set(vals).issubset(set(list(yn_map.keys()) + ["nan", ""])):
            candidate_cols.append(c)

for c in candidate_cols:
    df[c] = df[c].map(normalize_yes_no)

print("Normalized Yes/No columns:", candidate_cols)
```

Normalized Yes/No columns: ['self_employed', 'family_history', 'treatment', 'remote_work', 'tech_company', 'obs_co
nsequence']

I standardized Yes/No fields so all variations (e.g., “Y”, “TRUE”) become either Yes or No. This keeps categories consistent across the dataset.

Normalize gender-like columns:

```
gender_cols = [c for c in df.columns if c in ["gender", "sex"]]
print("Detected gender-like columns:", gender_cols)

def normalize_gender(val):
    if pd.isna(val):
        return np.nan
    s = str(val).strip().lower()

    male_terms = ["m", "male", "man", "cis male", "cis-man", "cisgender male", "male-ish", "mal"]
    female_terms = ["f", "female", "woman", "cis female", "cis-woman", "cisgender female", "femail"]
    nb_terms = ["non-binary", "nonbinary", "nb", "genderqueer", "trans", "agender", "fluid", "enby"]

    if any(t == s for t in male_terms) or "male" in s and "fe" not in s:
        return "Male"
    if any(t == s for t in female_terms) or "female" in s:
        return "Female"
    if any(t in s for t in nb_terms):
        return "Non-binary/Other"
    if s in ["prefer not to say", "na", "n/a", "none"]:
        return "Prefer not to say"
    # fallback
    return "Non-binary/Other"

for c in gender_cols:
    df[c] = df[c].map(normalize_gender)

if gender_cols:
    print("Gender value counts after normalization:")
    for c in gender_cols:
        print("\n\n", c)
        print(df[c].value_counts(dropna=False))
else:
    print("No gender-like columns found.")
```

Detected gender-like columns: ['gender']

Gender value counts after normalization:

```
\n gender
gender
Male          985
Female        249
Non-binary/Other  25
Name: count, dtype: int64
```

If a gender (or sex) column exists, I normalized common free-text entries into four categories: Male, Female, Non-binary/Other, and Prefer not to say. This reduces noisy labels while respecting inclusion.

V. EDA

Before Modeling

Goal :produce a clean, well-documented, model-ready table with an unambiguous **target**, sensible **encodings**, and **reproducible** preprocessing choices

Target: the outcome we predict (e.g., will disclose vs. won't).

Encodings: turning text/categorical answers into numbers (e.g., one-hot, ordinal).

Reproducible preprocessing: a pipeline (impute → encode → scale) that gives the same results every time you rerun it.

Target

- Predict whether someone **would disclose** a mental-health issue at work.

Yes” to any disclosure channel → target = 1
Otherwise → target = 0

4) Create target proxies (risk & disclosure)

```
1: import numpy as np
import pandas as pd

YES = ["yes", "y", "true", "1", "1", "1.0"]
NO = ["no", "n", "false", "f", "0", "0.0"]

def to_yes_no(x):
    """Map many truthy/falsy spellings to 'Yes'/'No'; keep common OSMI mid-answers."""
    if pd.isna(x):
        return np.nan
    s = str(x).strip().lower()
    if s in YES: return "Yes"
    if s in NO: return "No"
    if s in ["maybe", "some of them", "depends", "not sure", "unsure"]:
        return s.title() # 'Maybe', 'Some Of Them', 'depends', 'Not Sure', 'Unsure'
    return str(x) # keep original text if it's something else

def is_yes(x):
    if pd.isna(x): return False
    return str(x).strip().lower() in YES or str(x).strip().lower() == "yes"

cols_lc = [c.lower() for c in df.columns]
def has_col(substr):
    substr = substr.lower()
    return [orig for lc, orig in cols_lc.items() if substr in lc]

risk_candidates = []
for key in ["treat", "diagnos", "condition"]:
    risk_candidates += has_col(key)
risk_candidates = list(dict.fromkeys(risk_candidates)) # de-dup

risk_col = None
for c in risk_candidates:
    vals = df[c].dropna().astype(str).str.lower().unique()
    if any(v in YES or v in NO or v in ["yes", "no"] for v in vals):
        risk_col = c
        break
if risk_col is None and risk_candidates:
    risk_col = risk_candidates[0] # fallback: the first found

if risk_col is not None:
    df["has_condition"] = df[risk_col].map(to_yes_no)
else:
    df["has_condition"] = np.nan

disclose_cols = []
for key in ["supervisor", "coworker", "disclose", "interview"]:
    disclose_cols += has_col(key)
disclose_cols = list(dict.fromkeys(disclose_cols)) # de-dup

if disclose_cols:
    any_yes = np.zeros(len(df), dtype=bool)
    for c in disclose_cols:
        any_yes |= df[c].apply(is_yes).fillna(False).to_numpy()
    df["disclose_yes"] = np.where(any_yes, "Yes", "No")
else:
    df["disclose_yes"] = np.nan

df["has_condition_bin"] = df["has_condition"].map({"Yes":1, "No":0})
df["disclose_yes_bin"] = df["disclose_yes"].map({"Yes":1, "No":0})

print("Risk proxy column used:", risk_col)
print("Disclosure-related columns used (up to 8 shown):", disclose_cols[:8])

print("\nhas_condition value counts:")
print(df["has_condition"].value_counts(dropna=False))

print("\ndisclose_yes value counts:")
print(df["disclose_yes"].value_counts(dropna=False))

print("\nCrosstab (row=has_condition, col=disclose_yes):")
print(pd.crosstab(df["has_condition"], df["disclose_yes"], dropna=False))
```

Encodings (turn text into numbers)

One-hot Gender $\rightarrow$ [Female, Male, Non-binary] $\rightarrow$ [1/0, 1/0, 1/0]



Ordinal Leave $\rightarrow$ [Very easy ... Very difficult] $\rightarrow$ [1 ... 5]



Binary Anonymity $\rightarrow$ [Yes/No/Don't know] $\rightarrow$ [1/0/NA]

Reproducible preprocessing

Prevented leakage & locked a reproducible pipeline.

- **num_cols:** ['Age'] → 1 numeric feature (will be imputed + scaled).

make sure missing ages are filled in and the numbers are on a similar scale.

- **ord_cols:** ['leave_ord', 'work_interfere_ord'] → 2 ordinal features kept as integers

keep them as numbers in the right order ($1 < 2 < 3 < 4 < 5$), so the model knows which is bigger.

- **cat_cols:** 11 unordered categoricals will be one-hot encoded.

*The model can't read words, so turned each possible word into its own tiny light switch: **on = 1, off = 0**.*

*That's called **one-hot**.*

- **Transformed shape (5, 31):** with just 5 sample rows, your pipeline produced 31 model features total: 1 (Age) + 2 (ordinals) + 28 one-hot columns for the categories seen in those 5 rows.

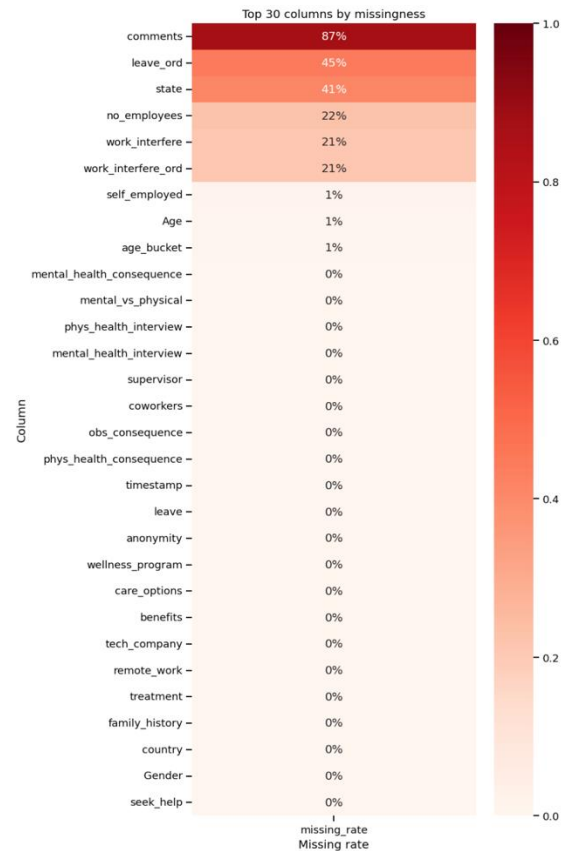
*Fed **5 people** (5 rows) into our “make-it-numbers” machine, and it came out with **31 switches/numbers** per person.*

where did 31 come from?

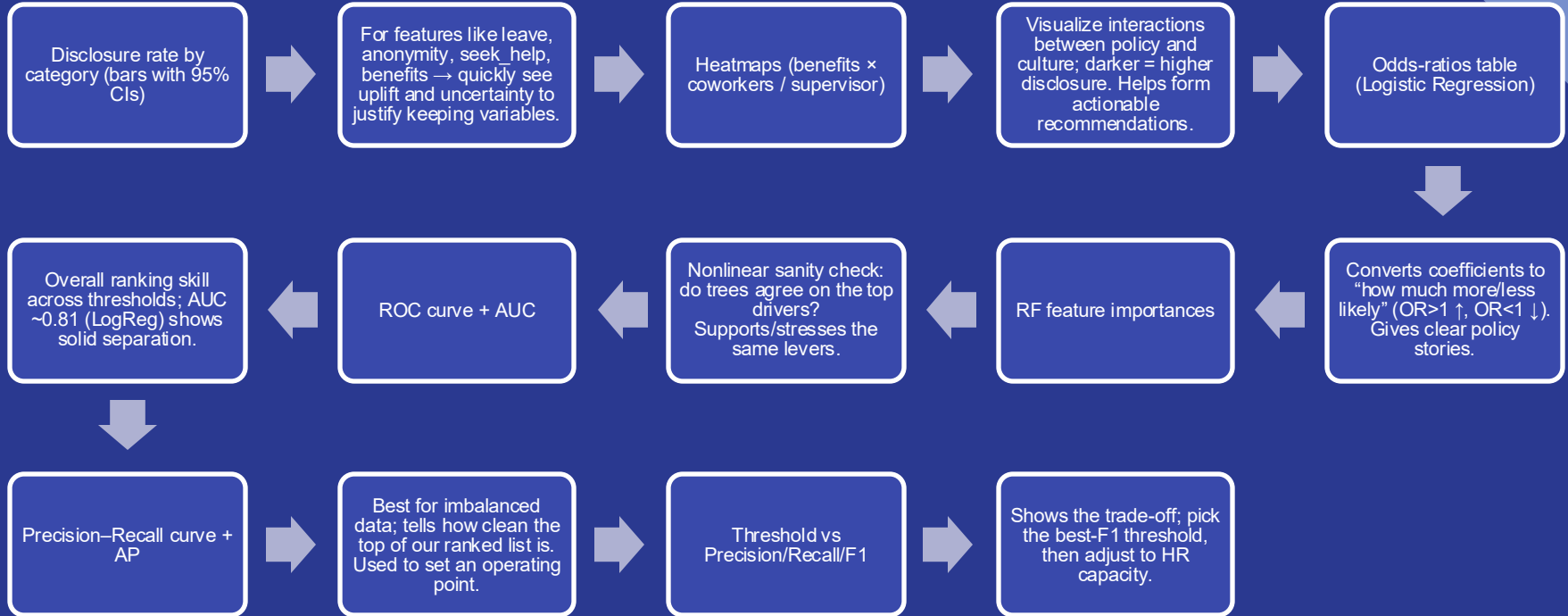
- 1 number for Age
- 2 ordered numbers (leave, work_interfere)
- **28 little on/off switches** for all the categories that appeared in those 5 rows
- **First 10 OHE names:** confirms one-hot worked and labels are clean (e.g., anonymity_Yes, benefits_No, ...). *labels for those little switches.*

Audited missingness & set imputation rules.

- Numeric: median imputation (and scaling for continuous).
- Ordinal: most-frequent imputation (keeping scale).
- Categorical: most-frequent imputation, with “Unknown” treated explicitly if helpful.



Why These Charts?



Policy / Awareness Variables

What variables I looked at

Leave (leave, leave_ord)

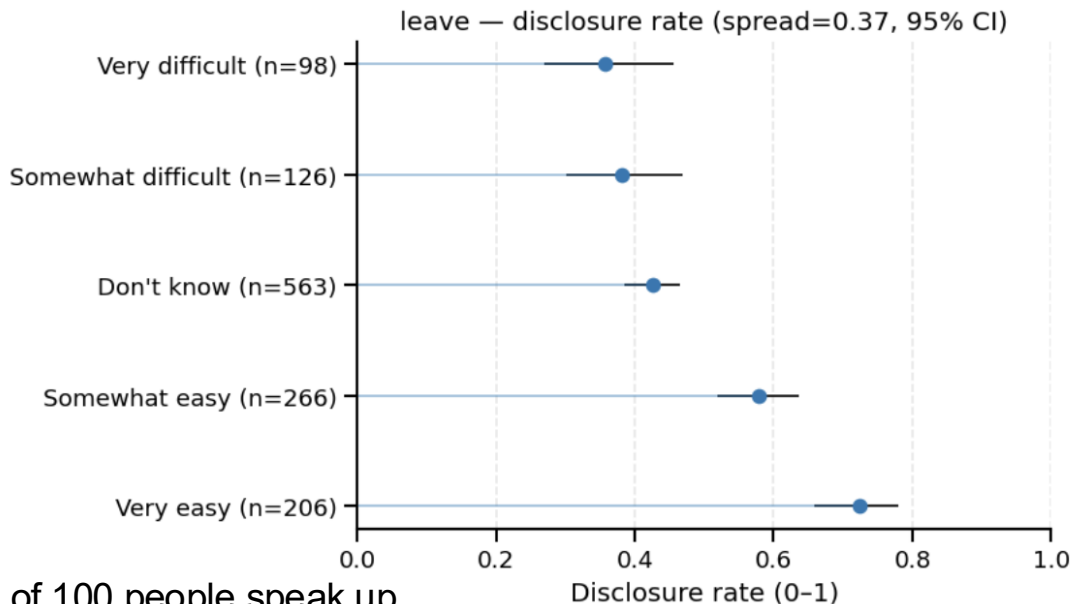
Very easy $\approx 74\%$ would disclose ($n=206$; *CI is wider because n isn't huge*)

Somewhat easy $\approx 59\%$ ($n=266$)

Don't know $\approx 43\%$ ($n=563$; *narrow CI—lots of respondents*)

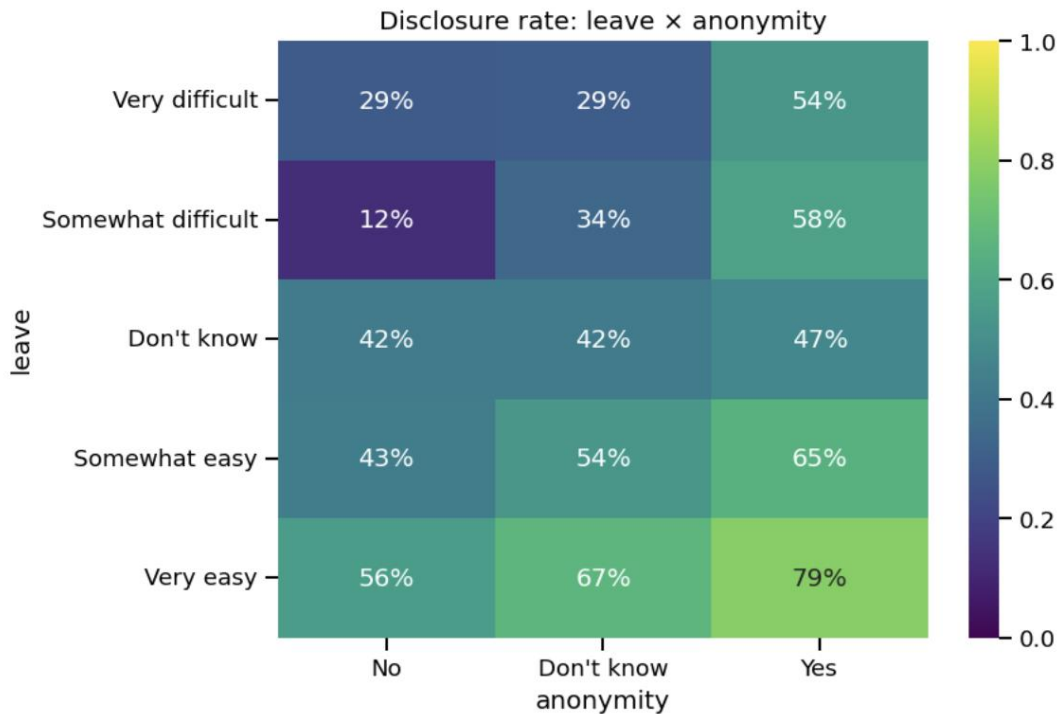
Somewhat difficult $\approx 39\%$ ($n=126$)

Very difficult $\approx 45\%$ ($n=98$; *wider CI*)



- When leave is very easy, about 74 out of 100 people speak up.
- When people don't know how leave works, only about 43 out of 100 speak up.
- When leave feels difficult, it's around 39–45 out of 100.

What variables help people speak up ?



- When leave is Very easy and there is anonymity → about **79 out of 100 tell**.
- When leave is Somewhat difficult and no anonymity → only **12 out of 100 tell**.

What variables I looked at

Anonymity

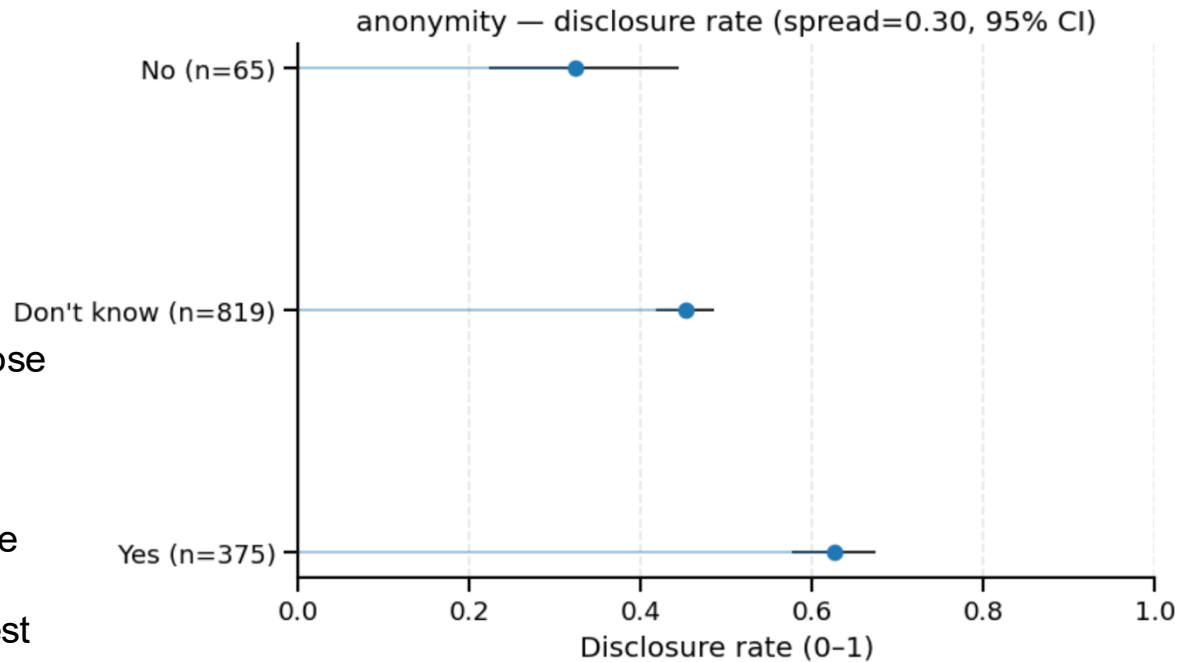
(anonymity): Employees disclose more when an **anonymous channel exists**—and even more than when they **aren't sure** it exists.

Yes, anonymity: ~64% would disclose (n=375)

Don't know: ~46% would disclose (n=819)

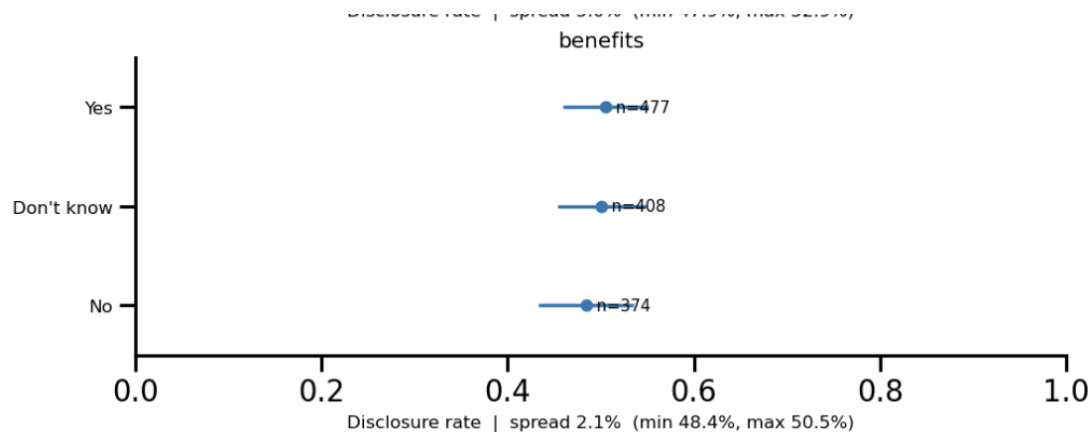
No anonymity: ~33% would disclose (n=65)

Gap (Yes – No): ~+31 points (largest separation after leave)



What variables I looked at

Benefits (benefits): Having MH benefits doesn't clearly change disclosure on its own.



Yes: ~50% disclose (n≈477)

Don't know: ~50% (n≈408)

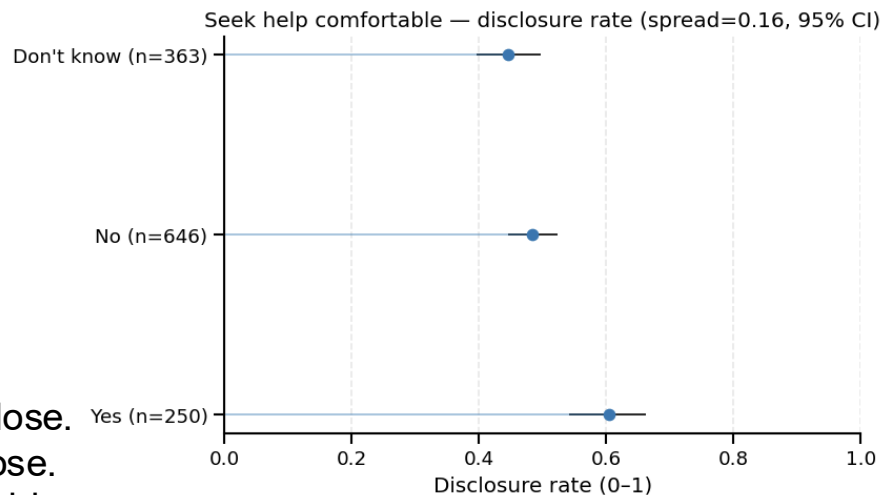
No: ~48% (n≈374)

“MH benefits by themselves don't move disclosure much; bigger levers are **leave** and **anonymity**.”

What variables I looked at

Care options (care_options): Comfort with help-seeking reflects **psychological safety** and awareness of how to get help.

Yes (n=250) → about 60 out of 100 would disclose.
No (n=646) → about 48 out of 100 would disclose.
Don't know (n=363) → about 45 out of 100 would disclose.
Spread ≈ 16 points (from ~45% to ~60%).

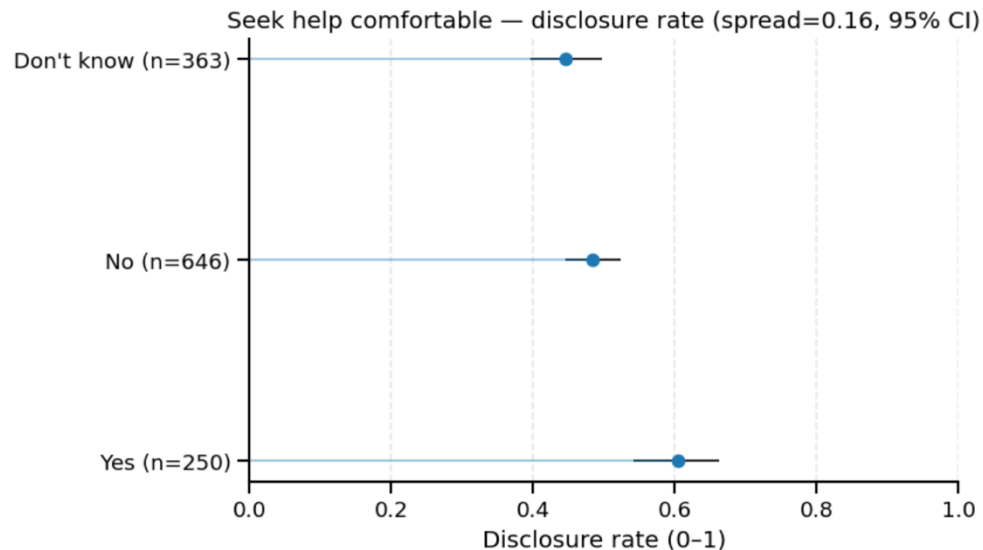


Comfort/Climate Variables

What variables I looked at

Seek help (seek_help):

Higher comfort → higher disclosure
“Don’t know” lowest + widest CI
(uncertainty)

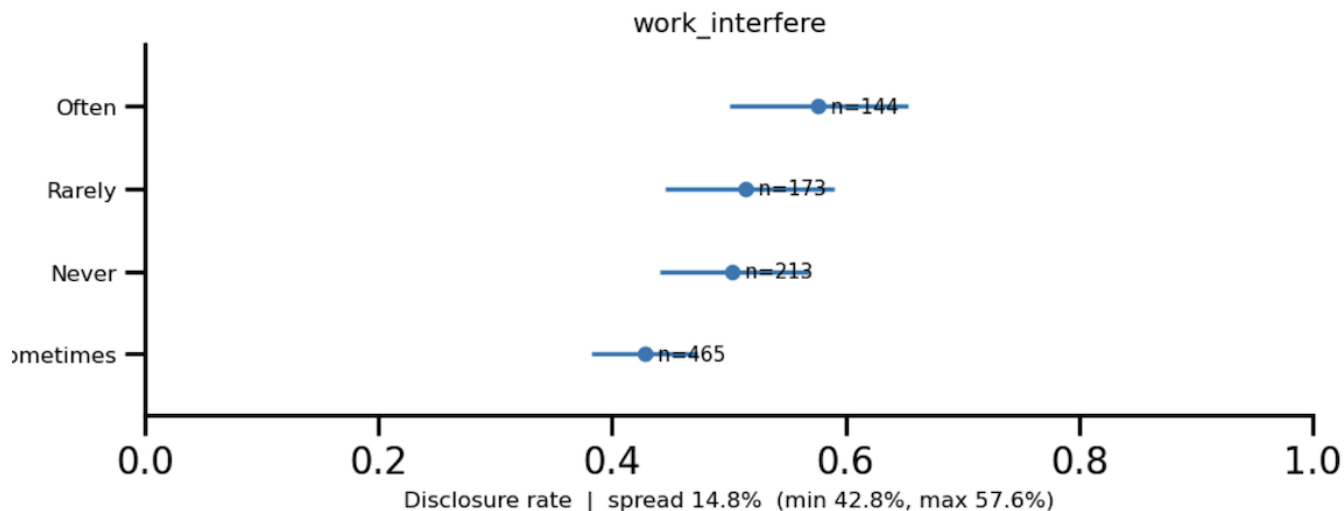


What variables I looked at

Work interfere (work_interfere, work_interfere_ord): Work interference shows a severity trend

Monotonic “dose-response”

Higher interference → lower disclosure

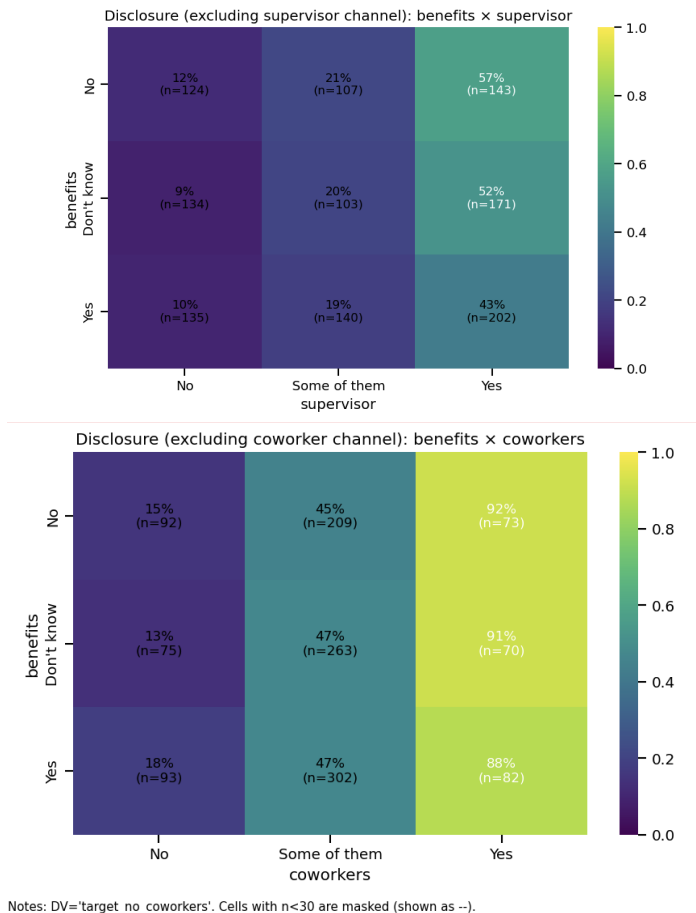
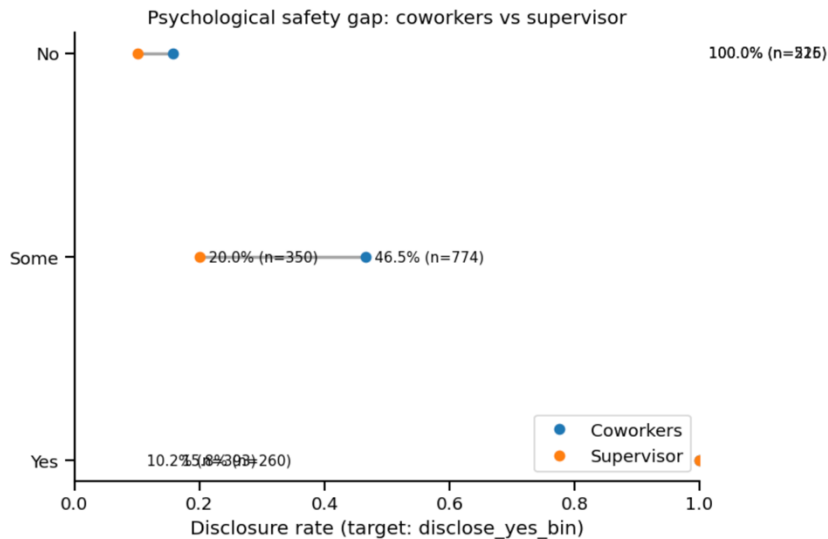


People/Org context Variables

What variables I looked at

Supervisor/Coworker(coworkers, supervisor):

Climate beats policy here: feeling safe with **supervisors or coworkers** drives disclosure much more than simply having benefits.

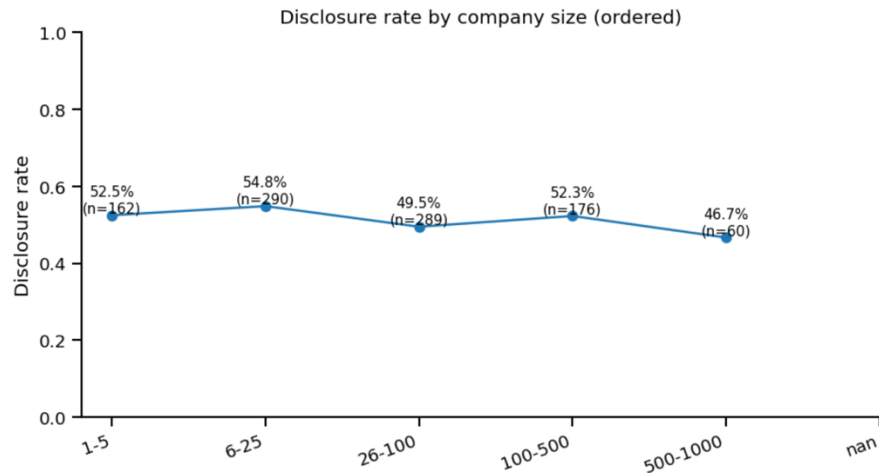


What variables I looked at

Company size (no_employees)

Profile by size band (shape:
flat/mono/U)

Effect smaller than
leave/anonymity

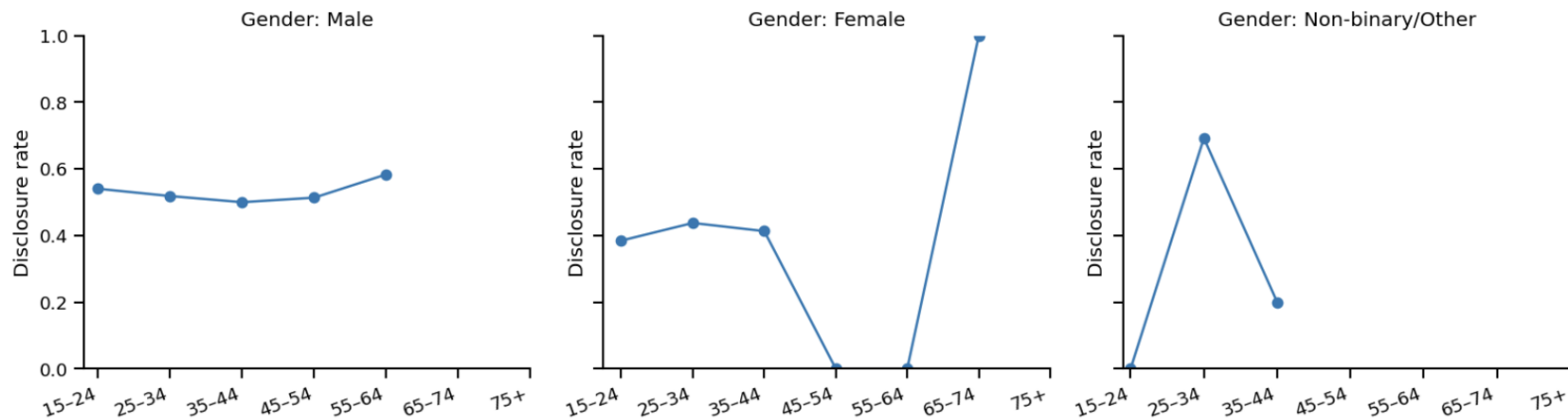


Demographics Variables

What variables I looked at

Age / age_bucket (Age, age_bucket)

Disclosure rate by age (10-year bins), by gender



- Modest differences across age
- Useful for audience profiling only

Why these diagnostics & visuals

- **Lollipop** (with 95% CIs) – shows the rate and the uncertainty; easy side-by-side category comparison.
- **Slopegraph** (coworkers vs supervisor) – reveals channel gaps at a glance.
- **Heatmaps** (policy × policy) – surface synergies (e.g., easy leave + anonymity).
- **Missingness heatmap** – shows data gaps and supports imputation choices.

Which variables I did **not** focus on (and why)

Target components (leakage risk)

Examples: supervisor, coworkers, mental_health_interview, phys_health_interview (the items used to build disclose_yes_bin).

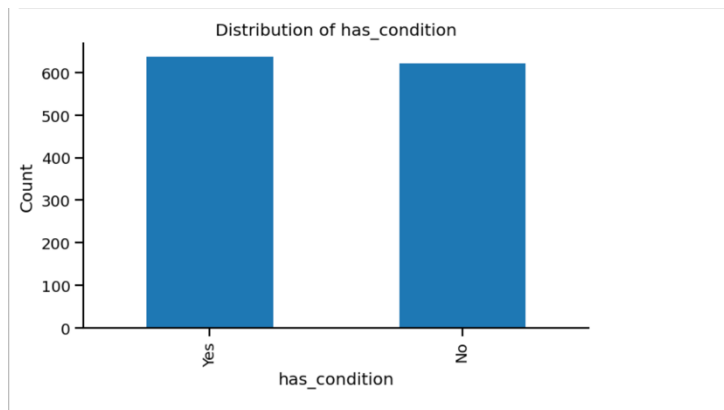
Why skip: Using them as features would **guarantee** higher disclosure when they're "Yes." I kept them for EDA visuals (with leakage-free targets) but **not** as predictors.

Which variables I did **not** focus on (and why)

Condition status (low signal)

Examples: treatment, has_condition, has_condition_bin.

Why skip: EDA showed ~50/50 disclosure whether condition = Yes or No (almost identical rates), so little predictive value; I kept it as context only.



Which variables I did **not** focus on (and why)

Free text / IDs / timestamps (not modelable / no hypothesis)

Examples: any **IDs**, raw **text** responses, raw **timestamps**; YEAR/WEEK used only to show volume over time.

Why skip: Not numeric or theoretically tied to disclosure; used YEAR/WEEK for **coverage plots** only, **not** as features.

Ultra-rare categories (unstable estimates)

Examples: tiny levels in Gender, country, or small subgroups elsewhere.

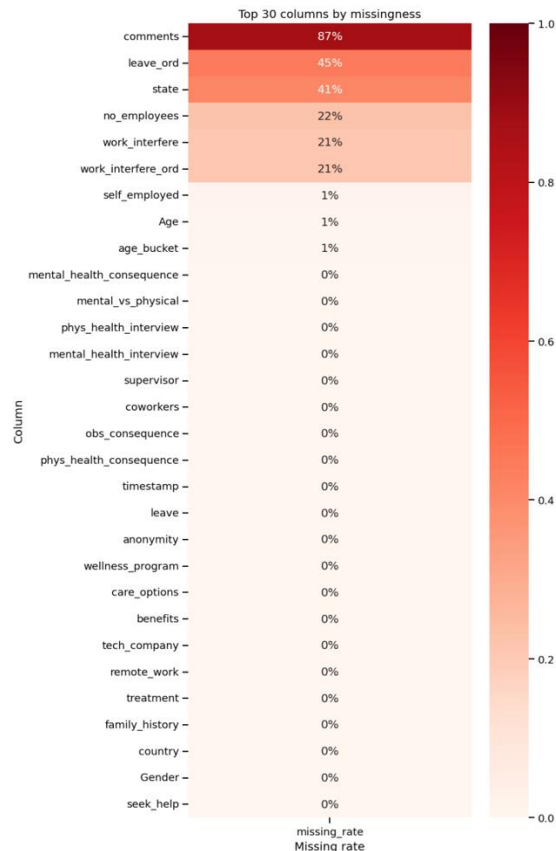
•**Why skip/collapse:** Very small **n** → noisy rates and unstable coefficients. I **collapsed** to “**Other**” or de-emphasized in charts.

Which variables I did **not** focus on (and why)

High-missing columns (data quality)

- Examples: any fields with **heavy missingness** in the missingness heatmap.

- **Why skip:** If not theoretically critical **and** hard to impute credibly, I excluded; otherwise I imputed (median or most-frequent) **inside the pipeline**.



What transformations I performed

What transformations I performed

- **Label cleanup:** normalized Yes/No/Don't know; merged "Some of them" → "Some".
- **Ordinal mapping:** leave (1–5), work_interfere (1–4) to **preserve order**.
- **Feature engineering:** age_bucket; no_employees set as an **ordered** category.
- **Binary helpers:** quick 0/1 versions (*_bin) for EDA/correlations (final model uses one-hot).
- **One-hot encoding:** unordered categories (anonymity, benefits, care_options, seek_help, no_employees, coworkers, supervisor, Gender, country, remote_work, wellness_program) → model-ready dummies with handle_unknown="ignore".
- **Imputation inside pipeline:** median (numeric), most-frequent (categorical/ordinal).
- **Scaling:** standardized true numeric (Age, stigma_index if used).
- **Leak-proof prep:** all of the above live in a **scikit-learn ColumnTransformer + Pipeline**, fit on **train only**.

VI. Feature Selection

Identifying Important Variables

Tier 1 (huge gaps)

- 1. Supervisor** — spread **0.90** → supportive managers massively lift disclosure.
- 2. Coworkers** — spread **0.84** → pro-disclosure peer culture matters almost as much.

Tier 2 (strong)

- 3) Leave** — spread **0.37** → clearer/easier leave correlates with more disclosure.
- 4) Anonymity** — spread **0.30** → anonymous channels help.

Tier 3 (moderate)

- 5) Seek_help** — spread **0.16**
- 6) Work interfere** — spread **0.15**
- 7) Wellness program** — spread **0.11**

Lower signal

- 8) No_employees** — **0.08**
- 9) Care_options** — **0.05**
- 10) Remote_work** — **0.03**
- 11) Benefits** — **0.02**

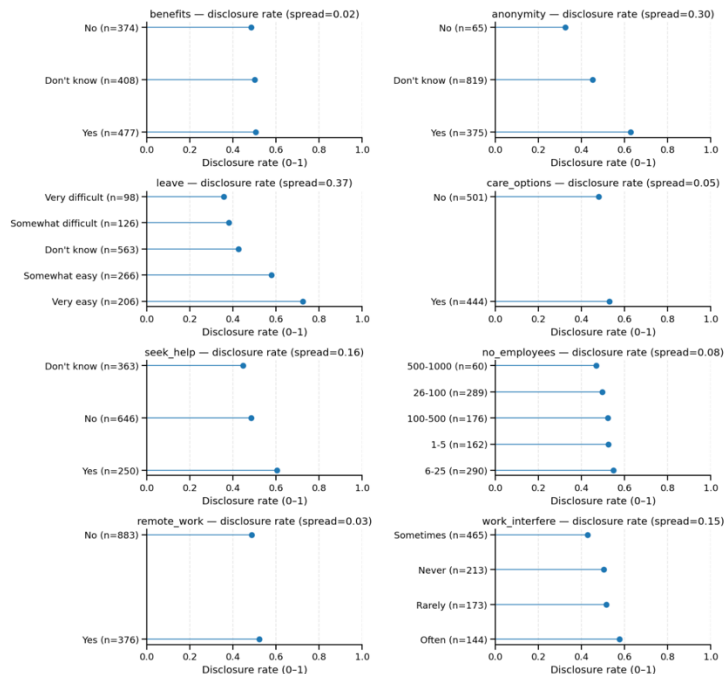
Methods Used



Univariate screening (rate gaps + 95% CI):

Comparing disclosure rates across single factors to spot big, reliable differences.

Why: fast signal of potential levers; surfaces large, stable differences.

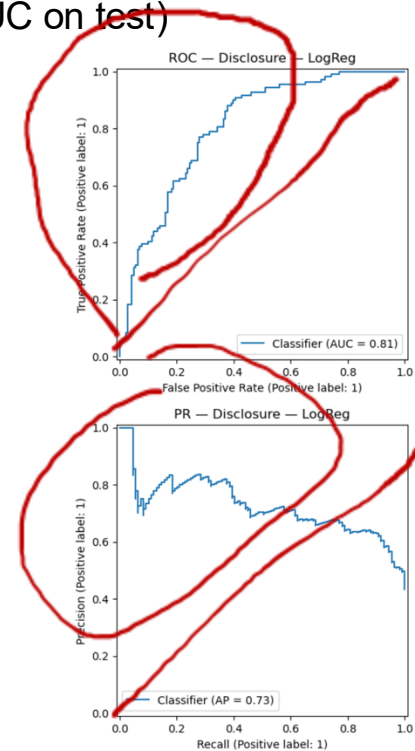


Feature Selection — Methods Used

We start with an interpretable **logistic regression** baseline with calibrated probabilities and many one-hot features.

- **Permutation importance** from the final model (drop in ROC-AUC on test)

- Interpretable baseline; calibrated probabilities; handles many OHE features.



VII. Modeling

Fundamental Concepts

- Supervised learning: trained on **labeled data** where the target is binary.
- (**disclose_yes_bin** / disclosure).
- **Task type: Classification** (not regression).

Build targets/proxies (robust disclosure, stigma, risk)

```
431: # == Targets & proxies ==  
  
def _map_yes_no(s):  
    s = s.astype(str).str.strip().str.lower()  
    return s.map({"yes":1, "no":0})  
  
def build_disclosure(df):  
    # Use coworkers/supervisor; try schemes and pick the one closest to ~50% overall  
    def map_str(x): return x.astype(str).str.strip().str.lower()  
    cw = map_str(df["coworkers"]) if "coworkers" in df else pd.Series(index=df.index, dtype=object)  
    sv = map_str(df["supervisor"]) if "supervisor" in df else pd.Series(index=df.index, dtype=object)  
  
    schemes = {}  
  
    # strict: only 'yes' counts  
    c = cw.eq("yes").astype(int); s = sv.eq("yes").astype(int)  
    schemes["strict"] = np.maximum(c, s)  
  
    # yes or "some of them" counts (both channels)  
    c = cw.isin(["yes", "some of them"]).astype(int); s = sv.isin(["yes", "some of them"]).astype(int)  
    schemes["yes_or_some_both"] = np.maximum(c, s)  
  
    # weighted: yes=1, some=0.5; positive if sum >= 1  
    c = cw.map({"yes":1, "some of them":0.5}).fillna(0).astype(float)  
    s = sv.map({"yes":1, "some of them":0.5}).fillna(0).astype(float)  
    schemes["weighted_sum_ge1"] = ((c + s) >= 1.0).astype(int)  
  
    # choose scheme closest to 0.497 (your earlier EDA)  
    target = 0.497  
    pick = min(schemes, key=lambda k: abs(schemes[k].mean() - target))  
    disc = schemes[pick].astype(int)  
    print((k: f"{v.mean():100:.1f}%" for k,v in schemes.items()))  
    print("Selected disclosure scheme:", pick, "| final rate:", f"{disc.mean():100:.1f}%")  
    return disc  
  
# Build disclosure (0/1)  
df["disclosure"] = build_disclosure(df)  
  
# Stigma index (0-1) from consequence items  
map_bin = {"yes":1, "no":0, "maybe":0.5, "sometimes":0.5, "often":1, "rarely":0.25, "don't know":0.5, "dont know":0.5}  
stigma_cols = ["mental_health_consequence", "phys_health_consequence", "mental_vs_physical", "obs_consequence"]  
parts = []  
for col in stigma_cols:  
    if col in df.columns:  
        parts.append(pd.to_numeric(df[col].astype(str).str.lower().map(map_bin), errors="coerce"))  
df["stigma_index"] = pd.concat(parts, axis=1).mean(axis=1)  
  
# Risk proxy (any condition/treatment/family history/interference)  
flags = []  
if "family_history" in df.columns:
```

Fundamental Concepts

- **Unsupervised learning:** *Not used.* I didn't do clustering or PCA in modeling—no groups discovered without labels.
- **Regression vs. Classification:** This project is **classification only** (predict 0/1 disclosure). I did **not** train ARIMA/SARIMA/SARIMAX or any numeric forecasters.

- **Supervised learning (Classification):** predict `disclose_yes_bin` (0/1).

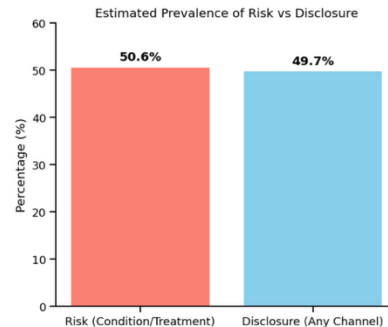
```
[262]: import matplotlib.pyplot as plt

proxies = {
    "Risk (Condition/Treatment)": 50.6,
    "Disclosure (Any Channel)": 49.7
}

fig, ax = plt.subplots(figsize=(6,5))
bars = ax.bar(proxies.keys(), proxies.values(), color=["salmon", "skyblue"])

# Add percentage labels on top of each bar
for bar in bars:
    height = bar.get_height()
    ax.text(bar.get_x() + bar.get_width()/2, height + 1,
            f"{height:.1f}%", ha="center", va="bottom", fontsize=12, fontweight="bold")

ax.set_title("Estimated Prevalence of Risk vs Disclosure")
ax.set_ylabel("Percentage (%)")
ax.set_ylim(0, 60) # give space for labels
plt.show()
```



Data Processing (Leak-Free & Reproducible)

- **Imputation (handle missing)**

- *Numeric*: **Median** (robust to outliers)
- *Categorical*: **Most frequent** (keeps mode/category intact)
- *Why*: Keep all rows without distorting distributions.

- **Scaling (standardization)**

- *Numeric*: **StandardScaler** → mean 0, std 1
- *Why*: Puts features on the same scale; stabilizes **Logistic Regression**; improves convergence and comparability.

- **Encoding (make categories numeric)**

- *Ordinal items*: **OrdinalEncoder** with **explicit order** (e.g., Very easy → ... → Very difficult)
- *Nominal items*: **One-Hot Encoding** with `handle_unknown=ignore`
- *Why*: Preserve real order where it exists; avoid fake order where it doesn't.

- **Leak Prevention**

- All steps inside a **scikit-learn Pipeline** and **fit on TRAIN only**, then applied to test.

- **Outcome**

- Clean, scaled, model-ready matrix; **interpretable coefficients** (LogReg odds ratios) and **stable performance** across models.

How did I pick my variables for modeling?



Imputation: numeric $\rightarrow$ median; categorical $\rightarrow$ most frequent.



Standardization: z-score scaling for numeric features.



Encoding: OHE for nominal; ordinal mapping for leave (add `work_interfere` next).



Pipelines: single leak-free preprocessor shared by LogReg (balanced) and RF (balanced).



Evaluation: ROC-AUC, PR-AP, F1, recall; threshold tuned for best F1.



Interpretation: LogReg odds ratios + RF feature importances.

Model Descriptions & Justifications

Logistic Regression (L2, class-balanced)

Why it fits my data: binary outcome; coefficients are interpretable; handles many one-hot features; you used class balancing to address skew.

Why this fits my data

- **Binary target:** disclose vs not disclose = perfect for logistic regression.
- **Interpretable:** coefficients tell direction (+/−) and strength after scaling.
- **Lots of categoricals:** one-hot plays nicely with logistic regression.
- **Imbalance handled:** `class_weight="balanced"` reduces bias toward the bigger class.
- **Stable optimizer:** `lbfgs` converges reliably; `max_iter` just gives it enough steps to finish.

```
In [45]: # == Preprocessing ==
def make_preprocess(num_cols, cat_cols):
    num_pipe = Pipeline([("imputer", SimpleImputer(strategy="median")),
                          ("scaler", StandardScaler())])
    cat_pipe = Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
                          ("ohe", OneHotEncoder(handle_unknown="ignore"))])
    return ColumnTransformer([("num", num_pipe, num_cols),
                              ("cat", cat_pipe, cat_cols)])

# == Model evaluator ==
def evaluate_models(preprocessor, X, y, label="Disclosure"):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE
    )
    models = {
        "LogReg": LogisticRegression(max_iter=2000, class_weight="balanced", solver="lbfgs"),
        "RF": RandomForestClassifier(n_estimators=400, random_state=RANDOM_STATE, class_weight="balanced")
    }
    results = {}
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    for name, clf in models.items():
        pipe = Pipeline([("pre", preprocessor), (name, clf)])
        cv_scores = cross_validate(
            pipe, X_train, y_train, cv=cv,
            scoring=["accuracy", "precision", "recall", "f1", "roc_auc"]
        )
        pipe.fit(X_train, y_train)
        y_prob = pipe.predict_proba(X_test)[:,1]
        y_pred = (y_prob >= 0.5).astype(int)
```

Logistic Regression: Odds Ratios (What drives disclosure?)

Biggest positive drivers: **very easy leave** and **seek_help_Yes** (culture/policy signals).

Biggest negative driver: **stigma_index** (as stigma rises, disclosure falls).

	feature	coef	odds_ratio	abs_coef
57	leave_Very easy	0.782552	2.187047	0.782552
1	stigma_index	-0.665042	0.514252	0.665042
63	seek_help_Yes	0.656411	1.927860	0.656411
25	gender_Neuter	0.636306	1.889489	0.636306
51	anonymity_No	-0.603926	0.546661	0.603926
2	gender_A little about you	0.573011	1.773599	0.573011
40	gender_p	0.573011	1.773599	0.573011
49	no_employees_More than 1000	-0.559366	0.571572	0.559366
41	gender_queer/she/they	0.520161	1.682298	0.520161
56	leave_Very difficult	-0.512824	0.598802	0.512824
26	gender_Trans-female	0.483389	1.621560	0.483389
37	gender_msle	-0.434260	0.647744	0.434260
20	gender_Male	-0.430788	0.649996	0.430788
52	anonymity_Yes	0.425976	1.531084	0.425976
44	no_employees_1-5	0.425128	1.529786	0.425128
19	gender_Male	0.398185	1.489120	0.398185
34	gender_malle	0.386270	1.471482	0.386270
62	seek_help_No	-0.379988	0.683869	0.379988

Model Descriptions & Justifications

Random Forest (class-balanced): clean/encode the data, then let **hundreds of decision trees** vote.

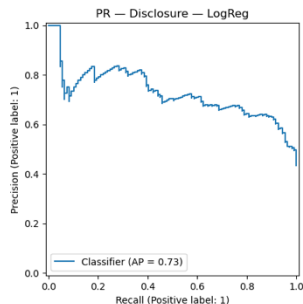
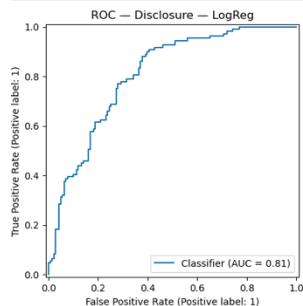
```
In [45]: # == Preprocessing ==
def make_preprocess(num_cols, cat_cols):
    num_pipe = Pipeline([("imputer", SimpleImputer(strategy="median")),
                          ("scaler", StandardScaler())])
    cat_pipe = Pipeline([("imputer", SimpleImputer(strategy="most_frequent")),
                          ("one", OneHotEncoder(handle_unknown="ignore"))])
    return ColumnTransformer([("num", num_pipe, num_cols),
                              ("cat", cat_pipe, cat_cols)])

# == Model evaluator ==
def evaluate_models(preprocessor, X, y, label="Disclosure"):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE
    )
    models = {
        "logit": LogisticRegression(max_iter=2000, class_weight="balanced", solver="lbfgs"),
        "RF": RandomForestClassifier(n_estimators=400, random_state=RANDOM_STATE, class_weight="balanced")
    }
    results = {}
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    for name, clf in models.items():
        pipe = Pipeline([("pre", preprocessor), (name, clf)])
        cv_scores = cross_validate(
            pipe, X_train, y_train, cv=cv,
            scoring=["accuracy", "precision", "recall", "f1", "roc_auc"]
        )
        pipe.fit(X_train, y_train)
        y_prob = pipe.predict_proba(X_test)[:,1]
        y_pred = (y_prob > 0.5).astype(int)
        # ROC & PR curves
        RocCurveDisplay.from_predictions(y_test, y_prob)
        plt.title(f"ROC - {label} - {name}"); plt.show()
        PrecisionRecallDisplay.from_predictions(y_test, y_prob)
        plt.title(f"PR - {label} - {name}"); plt.show()
    metrics = {
        "cv_accuracy": cv_scores["test_accuracy"].mean(),
        "cv_precision": cv_scores["test_precision"].mean(),
        "cv_recall": cv_scores["test_recall"].mean(),
        "cv_f1": cv_scores["test_f1"].mean(),
        "cv_roc_auc": cv_scores["test_roc_auc"].mean(),
        "test_accuracy": accuracy_score(y_test, y_pred),
        "test_precision": precision_score(y_test, y_pred),
        "test_recall": recall_score(y_test, y_pred),
        "test_f1": f1_score(y_test, y_pred),
        "test_roc_auc": roc_auc_score(y_test, y_prob),
        "confusion_matrix": confusion_matrix(y_test, y_pred),
    }
    results[name] = ("pipeline": pipe, "metrics": metrics, "y_test": y_test, "y_prob": y_prob, "y_pred": y_pred)
    print(f"({name}) - {label}")
    for k,v in metrics.items():
        if k != "confusion_matrix":
            print(f"({k}): {v:.3f}")
    print("confusion_matrix\n", metrics["confusion_matrix"])
    return results
```

- on-linear + interactions: trees naturally capture curved effects and feature interactions across many survey questions.
- Handles many categoricals: works well with one-hot features, no linearity assumptions.
- Robust baseline: tends to perform well without heavy tuning.
- Class imbalance addressed: `class_weight="balanced"` reduces bias toward the majority class.

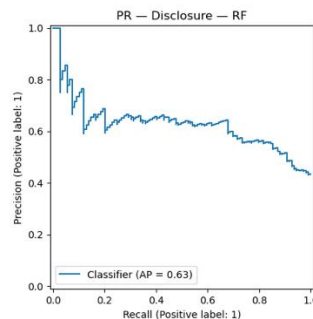
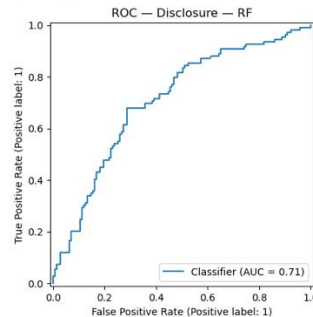
Why we chose Logistic Regression (F1 & Recall win)

Logistic Regression (L2, class-balanced)



```
LogReg - Disclosure
cv_accuracy: 0.667
cv_precision: 0.604
cv_recall: 0.677
cv_f1: 0.637
cv_roc_auc: 0.726
test_accuracy: 0.730
test_precision: 0.672
test_recall: 0.734
test_f1: 0.702
test_roc_auc: 0.806
confusion_matrix:
[[104 39]
 [ 29 80]]
```

Random Forest (class-balanced)



```
RF - Disclosure
cv_accuracy: 0.660
cv_precision: 0.625
cv_recall: 0.540
cv_f1: 0.578
cv_roc_auc: 0.699
test_accuracy: 0.659
test_precision: 0.619
test_recall: 0.550
test_f1: 0.583
test_roc_auc: 0.709
confusion_matrix:
[[106 37]
 [ 49 60]]
```

Results:

- LogReg**: F1 $\approx$ **0.70**, Recall $\approx$ **0.73**, ROC-AUC $\approx$ **0.81**, PR-AP $\approx$ **0.73**

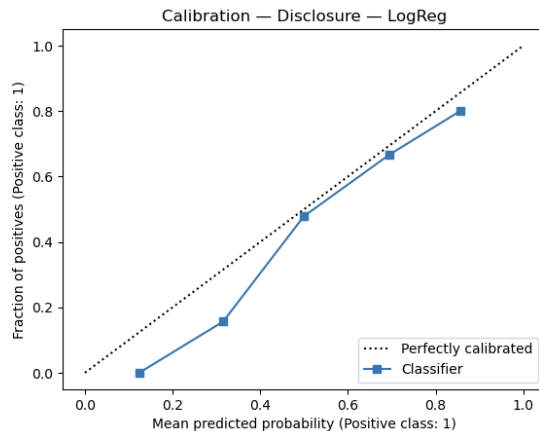
- RF**: F1 $\approx$ **0.58**, Recall $\approx$ **0.58**, ROC-AUC $\approx$ **0.71**, PR-AP $\approx$ **0.63**

Calibration curve — Logistic Regression (test set)

How did I you choose the threshold?”

I set the operating point by **maximizing F1** (balances precision & recall), and can **shift toward recall** if the program prioritizes catching more true disclosers.

- When the model says 70%, roughly 7 in 10 do disclose. We're **okay if a few extra get invited to support—much better than leaving someone out who needed it.**



Hyperparameter Tuning (just enough detail)

LogReg: tuned **C** (regularization strength) and **class_weight='balanced'**.



RF: tuned **n_estimators**, **max_depth**, **max_features** with class balancing.



Selection rule: pick config with best **CV F1**; confirm on test with **PR-AUC/ROC-AUC**.

Why other models were not appropriate

Why other models were not appropriate

•Time-series models (ARIMA / SARIMA / SARIMAX)

- Made for **forecasting a single number over time** (e.g., next week's sales).
- Our target is a **yes/no label** for each person, not a time-ordered value → **doesn't fit the problem**.

•Linear Regression

- Predicts a **continuous number**; our target is **0/1**.
- Would give uncalibrated numbers outside 0–1 and is **not a classifier**.

•KNN / Naive Bayes

- **KNN** struggles with many one-hot features and mixed data; slow at prediction.
- **Naive Bayes** assumes features are independent (not true for our survey variables) → weaker accuracy.

•SVM

- Can work, but needs careful tuning and kernels; **harder to explain** to non-technical stakeholders and can be slower on many features.

•Gradient Boosting / XGBoost / Neural Nets

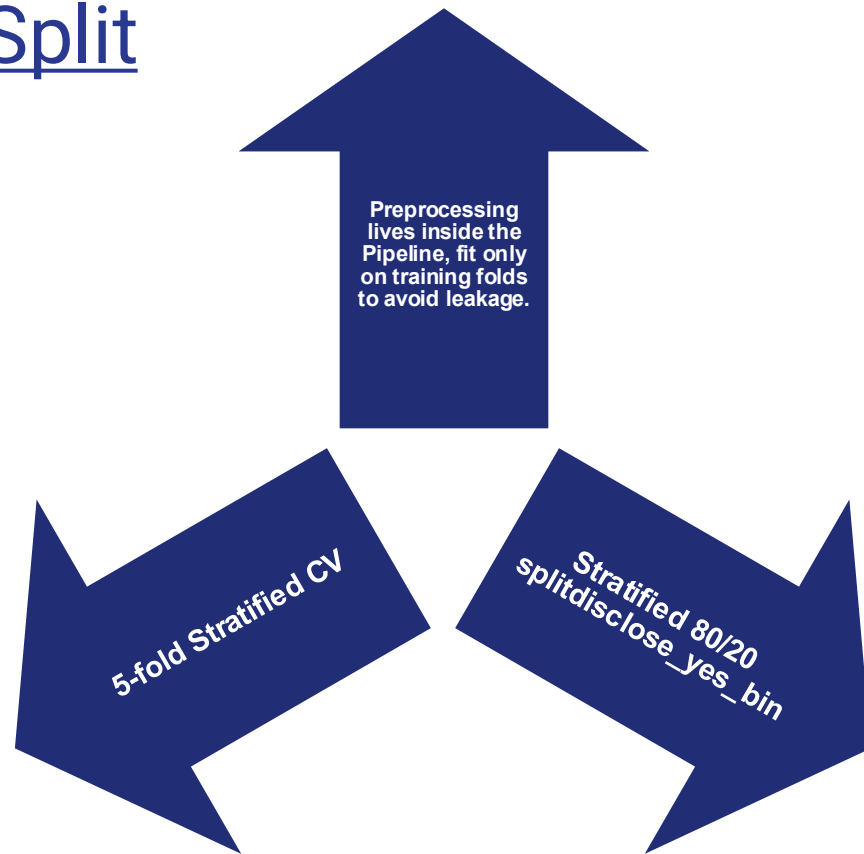
- Powerful but **more complex** to tune and **less interpretable** out-of-the-box.
- Our simpler models already deliver **good performance** (AUC 0.81, AP 0.73) with **clear explanations** and **calibrated probabilities**.

•Unsupervised methods (clustering, PCA)

- Find structure **without labels**; useful for EDA segments, not for a **labeled prediction** target like disclosure.

VIII. Evaluation

Train-Test Split



Threshold & Confusion Matrix

Operating point chosen by maximizing F1 on the validation curve

At the chosen threshold, compute and display the confusion matrix to show Precision/Recall on the positive class.

Confusion Matrix = count of hits and misses at that cutoff:

TP (True Positive): we said "yes," and they did disclose.

FP (False Positive): we said "yes," but they didn't.

TN (True Negative): we said "no," and they didn't disclose.

FN (False Negative): we said "no," but they did disclose.

Additional Model Diagnostics

- Calibration curve: probabilities are well-calibrated in the decision range (scores ~ 0.5 – 0.85 track observed disclosure rates closely).
- Permutation importance (test set): measures drop in ROC-AUC when a feature is shuffled, tying “importance” to actual predictive lift on held-out data.

Metrics (test set)

ROC-AUC: 0.81 → strong ability to rank disclosers above non-disclosers across thresholds.

PR / Average Precision: 0.73 → solid precision–recall trade-off for the positive class (“will disclose”).

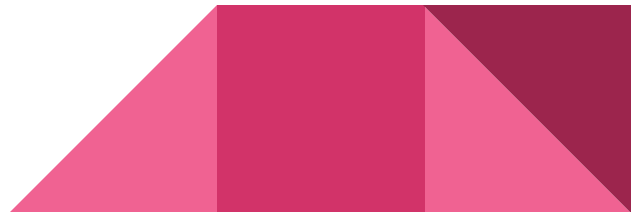
Curves shown: ROC and PR with metrics in the legends.

Operating point: choose threshold by maximizing F1; report confusion matrix at that threshold.

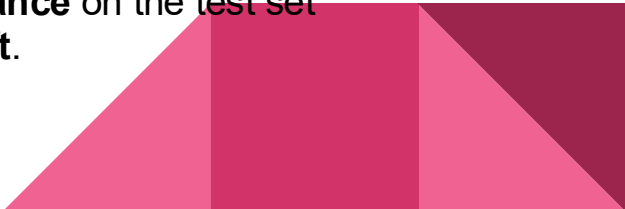
Calibration: probabilities are well-calibrated in the decision range (0.5–0.85).

Threshold & Confusion Matrix

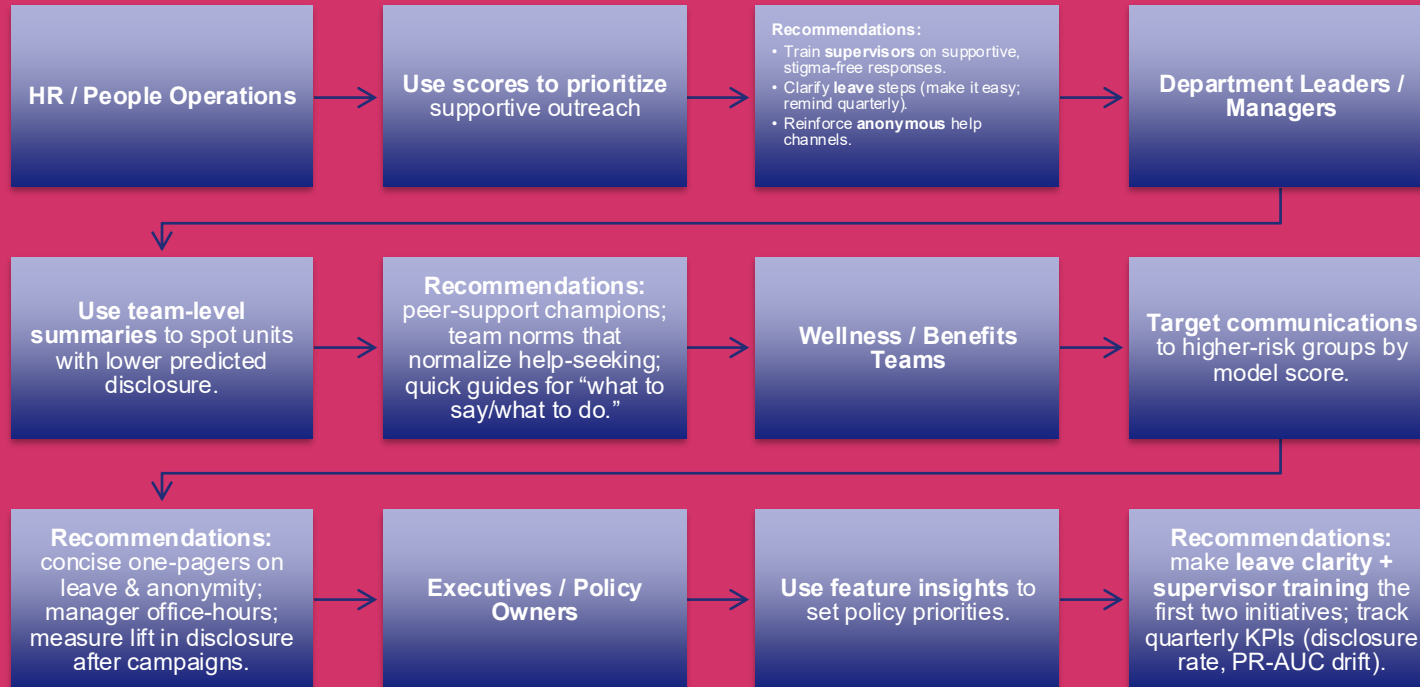
- Operating point chosen by maximizing F1 on the validation curve
- At the chosen threshold, compute and display the confusion matrix to show Precision/Recall on the positive class.



What I learned about the data and the problem.

- Disclosure is predictable enough to help decisions
 - **Workplace climate matters most: Supervisor and Coworker support show the strongest association**
 - **Leave** (ease/clarity) shows a **monotonic drop**—harder leave → lower disclosure.
 - **Some levers are weaker: Anonymity / seek_help / work_interfere show small but meaningful effects;**
 - **Benefits, company size, age, remote work show little signal in this sample.**
 - **Model-based explanation agrees with EDA: Permutation importance on the test set aligns with these patterns, tying “importance” to actual predictive lift.**
- 

Who may be interested, and what recommendations would I make?



What more would I want to do with the data?

Done

- Cleaned/encoded data; leakage-safe Pipelines; stratified CV + test; ROC/PR, calibration, and permutation importance; identified top policy levers.

Next steps

- Fairness: check precision/recall & calibration by subgroup (e.g., role, region).
- Stability: compute permutation importances across folds (mean $\pm$ SD ranks).
- Policy “what-if”s: simulate impact of improving leave or supervisor support on predicted disclosure.
- Outcome linkage: if available, test whether higher disclosure correlates with retention/engagement.
- Dashboard: simple internal tool to monitor threshold metrics and model drift.

Take-Home Message

Demonstrates the power of supervised classification to model and predict workplace **mental-health disclosure** using labeled survey data.

A light blue downward-pointing arrow indicating the flow from the first step to the second.

Identifying key patterns—**supervisor** and **coworker** support and **leave clarity/ease**—that most influence disclosure.

A light blue downward-pointing arrow indicating the flow from the second step to the third.

Applying appropriate, calibrated models (e.g., **Logistic Regression** and **Random Forest**) provides actionable insights for **HR and leaders** to target support, improve policies, and better prepare teams to respond to employees' needs.



Thank You!